

데이터분석_텍스트 마이닝

정규표현식 정리_260428

1. 정규표현식 기본 메타 문자

(1) 문자 매칭 관련 문법

문법	의미	예시	설명
.	임의의 한 문자 (줄바꿈 \n 제외)	a.b $\Rightarrow$ aab, a0b	a·b 사이 아무 문자 1개 있으면 매치
[]	문자 집합	[abc] $\Rightarrow$ a, b, c	대괄호 안 문자 중 하나 매치
[^]	부정 문자 집합	[^abc] $\Rightarrow$ d, 1	a, b, c가 아닌 문자 매치
[a-z]	범위 지정	[a-z]+ $\Rightarrow$ python	소문자 알파벳 1개 이상
[0-9]	숫자 범위	[0-9]+ $\Rightarrow$ 123	숫자 1개 이상
\d	숫자	\d+ $\Rightarrow$ 2026	$\equiv$ [0-9]
\D	숫자 아닌 문자	\D+ $\Rightarrow$ abc	숫자가 아닌 문자
\s	공백 문자	\s+ $\Rightarrow$ 공백, 탭	space, tab, newline 등
\S	공백 아닌 문자	\S+ $\Rightarrow$ python	공백 아닌 문자
\w	문자·숫자·밑줄	\w+ $\Rightarrow$ abc_123	영문자, 숫자, _
\W	\w 아닌 문자	\W+ $\Rightarrow$!@#	특수문자 등

※ a\b $\Rightarrow$ 문자 그대로의 점 유지

(2) 반복 관련 문법

문법	의미	예시	설명
*	앞 문자 0회 이상 반복	ca*t $\Rightarrow$ ct, cat, caaat	^a $\Rightarrow$ 없어도 되고 여러 번 있어도 됨
+	앞 문자 1회 이상 반복	ca+t $\Rightarrow$ cat, caaat	^a $\Rightarrow$ 최소 1번은 있어야 함
?	앞 문자 0회 또는 1회	ab?c $\Rightarrow$ abc, ac	^b $\Rightarrow$ 있어도 되고 없어도 됨
{m}	m회 반복	ca{2}t $\Rightarrow$ caat	a $\Rightarrow$ 정확히 2번
{m,n}	m회 이상 n회 이하 반복	ca{2,5}t $\Rightarrow$ caat, caaaat	a $\Rightarrow$ 2~5회
{m,}	m회 이상 반복	a{3,} $\Rightarrow$ aaa, aaaa	최소 3회 이상
{,n}	n회 이하 반복	a{,3} $\Rightarrow$ "", a, aa, aaa	최대 3회 이하

※ 동의어:
 {1,} = +
 {0,} = *
 {0,1} = ?

2. 위치 지정 메타 문자

문법	의미	예시	설명
<code>^</code>	문자열의 시작	<code>^Life ⇒ Life is too short</code>	문자열 시작 ⇒ Life必
<code>\$</code>	문자열의 끝	<code>short\$ ⇒ Life is too short</code>	문자열 끝 ⇒ short必
<code>\A</code>	전체 문자열의 시작	<code>\ALife ⇒ Life is too short</code>	<code>^</code> 유사 + MULTILINE 영향 상이
<code>\Z</code>	전체 문자열의 끝	<code>short\Z ⇒ Life is too short</code>	<code>\$</code> 유사 + 전체 문자열 끝 기준
<code>\b</code>	단어 경계	<code>r'\bclass\b' ⇒ no class at all</code>	독립된 단어(class)만 매치
<code>\B</code>	단어 경계가 아님	<code>r'\Bclass\B' ⇒ declassified內→class</code>	단어 내부에 있는 경우 매치

※ ex. `r'\bclass\b' → class` 독립된 단어에는 매치,
 다른 단어 안에 포함된 경우(`declassified·subclass`) → 매치X
 cf) `\B →` 단어 경계가 아닌 위치에서만 매치

3. 선택과 그룹 문법

(1) 문자 매칭 관련 문법

문법	의미	예시	설명
<code>`A</code>	<code>B`</code>	A 또는 B	<code>`Crow</code>

```
import re
p = re.compile('Crow|Servo')
m = p.match('CrowHello')
print(m.group()) # Crow
```

(2) 그룹 문법

문법	의미	예시	설명
<code>()</code>	그룹 지정	<code>(ABC)+</code>	ABC 전체를 하나의 반복 단위로 처리
<code>group(0)</code>	전체 매치 문자열	<code>m.group(0)</code>	매치된 전체 문자열
<code>group(1)</code>	첫 번째 그룹	<code>m.group(1)</code>	첫 번째 괄호에 해당하는 문자열
<code>group(2)</code>	두 번째 그룹	<code>m.group(2)</code>	두 번째 괄호에 해당하는 문자열
<code>(?P<이름>...)</code>	이름 있는 그룹	<code>(?P<name>\w+)</code>	그룹에 이름을 붙여 참조
<code>\1, \2</code>	그룹 재참조	<code>r'(\b\w+)\s+\1'</code>	같은 단어가 반복되는 경우 탐색
<code>(?P=이름)</code>	이름 있는 그룹 재참조	<code>(?P=word)</code>	이름으로 그룹 재참조

```
import re

p = re.compile(r"(\w+)\s+(\d+[-]\d+[-]\d+)")
m = p.search("park 010-1234-1234")

print(m.group(0)) # park 010-1234-1234
print(m.group(1)) # park
print(m.group(2)) # 010-1234-1234
```

※ 그룹: 반복 처리(by. 여러 문자 하나의 단위)
매치된 문자열 중 원하는 부분만 추출

4. 전방 탐색:

특정 조건이 뒤에 있는지 확인 → 그 조건 자체는 최종 결과에 포함X

문법	이름	의미	예시
(?=...)	긍정형 전방 탐색	뒤에 특정 패턴 → 매치	.+(?=): http://google.com ⇒ http
(?!...)	부정형 전방 탐색	뒤에 특정 패턴X → 매치	*[.](?!bat\$).* \$ ⇒ .bat 아닌 파일명

```
import re

p = re.compile(".*+(?=)")
m = p.search("http://google.com")
print(m.group()) # http
```

※ ":" ⇒ 실제 결과에는 포함X

5. Greedy와 Non-Greedy

구분	문법	의미	예시	결과
Greedy	.*	가능한 한 많이 매치	<.*>	<html><head><title>Title</title>
Non-Greedy	.*?	가능한 한 적게 매치	<.*?>	<html>
Non-Greedy 형태	*?, +?, {m,n}?	최소 반복	.+?	필요한 만큼만 매치

```
import re

s = '<html><head><title>Title</title>'

print(re.match('<.*>', s).group())
# <html><head><title>Title</title>

print(re.match('<.*?>', s).group())
# <html>
```

※ 정규표현식의 반복 문자: 가능한 한 긴 문자열을 매치(greedy 매칭)
cf) ? + 반복 기호 뒤 ⇒ 최소한으로 매치(non-greedy 방식)

6. 파이썬 re 라이브러리 주요 함수

(1) 기본 검색 함수

함수·메서드	기능	반환값	특징
<code>re.compile(pattern)</code>	정규식⇒to.패턴 객체	Pattern 객체	같은 정규식 반복 사용할 때 효율적
<code>match()</code>	매치 여부 확인 (from.문자열 처음)	Match 객체·None	시작 위치가 맞아야 함
<code>search()</code>	첫 매치 탐색 (in. 문자열 전체)	Match 객체·None	중간에 있어도 찾음
<code>findall()</code>	매치되는 모든 문자열 출력	리스트	결과 문자열만 필요할 때 사용
<code>finditer()</code>	매치되는 모든 결과 출력	iterator	각 결과의 위치 정보까지 필요

```
import re

p = re.compile('[a-z]+')

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
m = re.match(p, "python")

# 2. 정규식 문자열을 패턴 객체로 컴파일
m = p.match("python")

# 3. 결과 확인
print(m)
print(m.group())
<re.Match object; span=(0, 6), match='python'>
python
```

(2) match()와 search() 비교

구분	match()	search()
검색 위치	only. 맨 앞부터	문자열 전체를 검색
실패 예시	p.match("3 python") ⇒ None	p.search("3 python") ⇒ python 매치
사용 상황	특정 패턴으로 시작하는지 확인	특정 패턴이 포함되는지 확인
반환값	Match 객체·None	Match 객체·None

```
import re

p = re.compile('[a-z]+')

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
print(re.match(p, "3 python"))      # None
print(re.search(p, "3 python"))     # <re.Match object; match='python'>

# 2. 정규식 문자열을 패턴 객체로 컴파일
print(p.match("3 python"))          # None
print(p.search("3 python"))         # <re.Match object; match='python'>
```

(3) findall()과 finditer() 비교

구분	findall()	finditer()
반환 형태	리스트	반복 가능한 객체
반환 내용	매치된 문자열	Match 객체
장점	결과 확인이 간단	위치 정보, 그룹 정보 확인 가능
예시	['life', 'is', 'too', 'short']	Match 객체 여러 개

```
import re

p = re.compile('[a-z]+')

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
print(re.findall(p, "life is too short"))# ['life', 'is', 'too', 'short']
for m in re.finditer(p, "life is too short"):
    print(m.group(), m.span())
#life (0, 4)
#is (5, 7)
#too (8, 11)
#short (12, 17)
```

```
# 2. 정규식 문자열을 패턴 객체로 컴파일
print(p.findall("life is too short"))# ['life', 'is', 'too', 'short']
for m in p.finditer("life is too short"):
    print(m.group(), m.span())
#life (0, 4)
#is (5, 7)
#too (8, 11)
#short (12, 17)
```

7. Match 객체 주요 메서드

메서드	기능	예시 결과
group()	매치된 문자열 출력	'python'
start()	매치 시작 위치 출력	2
end()	매치 끝 위치 출력	8
span()	튜플(시작, 끝) 출력	(2, 8)

```
import re

p = re.compile('[a-z]+')

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
m = re.search(p, "3 python")

# 2. 정규식 문자열을 패턴 객체로 컴파일
m = p.search("3 python")

# 출력 결과(동일)
print(m.group()) # python
print(m.start()) # 2
print(m.end()) # 8
print(m.span()) # (2, 8)
```

※ Match: “무엇이 매치” + “어느 위치에서 매치”까지 확인

8. 문자열 바꾸기: sub()와 subn()

함수	기능	반환값	예시
sub()	매치된 부분을 다른 문자열로 치환	변경된 문자열	blue → colour
sub(count=1)	지정 횟수만큼만 치환	변경된 문자열	첫 번째 매치만 변경
subn()	치환 결과와 치환 횟수 반환	(변경 문자열, 횟수)	('colour socks...', 2)

```
import re

p = re.compile('(blue|white|red)')

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
print(re.sub(p, 'colour', 'blue socks and red shoes'))# colour socks and colour shoes
print(re.sub(p, 'colour', 'blue socks and red shoes', count=1))# colour socks and red shoes
print(re.subn(p, 'colour', 'blue socks and red shoes'))# ('colour socks and colour shoes', 2)

# 2. 정규식 문자열을 패턴 객체로 컴파일
print(p.sub('colour', 'blue socks and red shoes'))# colour socks and colour shoes
print(p.sub('colour', 'blue socks and red shoes', count=1))# colour socks and red shoes
print(p.subn('colour', 'blue socks and red shoes'))# ('colour socks and colour shoes', 2)
```

9. sub()에서 그룹 참조 사용

방식	문법	의미
이름 있는 그룹 참조	Wg<name>	그룹 이름으로 치환에 활용
번호 그룹 참조	Wg<1>, Wg<2>	그룹 번호로 치환에 활용
함수 사용	sub(함수, 문자열)	매치된 값마다 함수를 적용해 치환

```
import re

p = re.compile(r"(?P<name>\w+)\s+(?P<phone>(\d+)[-]\d+[-]\d+)")

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
result = re.sub(p, r"\g<phone> \g<name>", "park 010-1234-1234")

# 2. 정규식 문자열을 패턴 객체로 컴파일
result = p.sub(r"\g<phone> \g<name>", "park 010-1234-1234")

# 출력 결과(동일)
print(result)# 010-1234-1234 park
```

```
import re

def hexrepl(match):
    value = int(match.group())
    return hex(value)

p = re.compile(r'\d+')

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
result = re.sub(p, hexrepl, 'Call 65490 for printing, 49152 for user code.')

# 2. 정규식 문자열을 패턴 객체로 컴파일
result = p.sub(hexrepl, 'Call 65490 for printing, 49152 for user code.')

# 출력결과(동일)
print(result)# Call 0xffd2 for printing, 0xc000 for user code.
```

10. 컴파일 옵션

옵션	약어	기능	사용 예시
re.DOTALL	re.S	.이 줄바꿈 문자 \n까지 매치	여러 줄 텍스트 전체 검색
re.IGNORECASE	re.I	대소문자 구분 없이 매치	python, Python, PYTHON 모두 매치
re.MULTILINE	re.M	^, \$를 각 줄의 시작·끝에 적용	여러 줄에서 줄별 검색
re.VERBOSE	re.X	정규식을 여러 줄로 작성하고 주석 사용 가능	복잡한 정규식 가독성 향상

```
import re

p = re.compile('a.b', re.DOTALL)

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
print(re.match(p, 'a\nb'))# <re.Match object; span=(0, 3), match='a\nb'>

# 2. 정규식 문자열을 패턴 객체로 컴파일
print(p.match('a\nb'))# <re.Match object; span=(0, 3), match='a\nb'>

# 줄바꿈도 .에 매치됨
```



```
import re

p = re.compile('[a-z]+', re.I)

# 1. re 함수에 컴파일된 객체 p를 전달하는 방식
print(re.match(p, 'PYTHON'))# <re.Match object; span=(0, 6), match='PYTHON'>

# 2. 정규식 문자열을 패턴 객체로 컴파일
print(p.match('PYTHON'))# <re.Match object; span=(0, 6), match='PYTHON'>

# 대소문자 구분 없이 매치
```

※ DOTALL, IGNORECASE, MULTILINE, VERBOSE ⇒ 정규식의 해석 방식을 바꾸는 대표 옵션
 MULTILINE: ^, \$ ⇒ “전체 문자열의 시작·끝” → “각 줄의 시작·끝”으로 확장할 때 중요

11. Raw String과 역슬래시 문제

작성 방식	의미	권장 여부
'WWsection'	의도와 다르게 해석 여지(by.파이썬 문자열 규칙)	주의 필요
'WWWWsection'	WWsection 전달(in. 정규식 엔진)	가능하지만 복잡
r'WWsection'	raw string 사용	권장

```
import re

p = re.compile(r'\\section')
```

※ in. 정규표현식 ⇒ raw string 사용 권장(r'...')
 + for. 역슬래시 문제 해결 → raw string 표현 권장